Name: Achi Vaishnavi

Hall Ticket:2303A51554

Batch:29

Assessment:13.5

Task 1: Refactoring – Removing Global Variables

Prompt:

Refactor the code to remove unnecessary global variables.Pass values using function parameters instead of relying on global state.Improve modularity and testability.

Code:

```
"""def calculate_area(radius):
    Calculate the area of a circle given its radius.
    import math
    return math.pi * radius ** 2
def calculate_circumference(radius):
    Calculate the circumference of a circle given its radius.
    import math
    return 2 * math.pi * radius
def main():
    radius = float(input("Enter the radius of the circle: "))
    area = calculate_area(radius)
    circumference = calculate_circumference(radius)
    print(f"The area of the circle is: {area:.2f}")
    print(f"The circumference of the circle is: {circumference:.2f}")
if __name__ == "__main__":
    main()
"""
```

Output:

```
Enter the radius of the circle: 2
The area of the circle is: 12.57
The circumference of the circle is: 12.57
```

Significance:

Global variables create tight coupling and make testing difficult. Passing values through function parameters improves modularity, reduces hidden dependencies, and makes functions reusable and predictable.

Task 2: Refactoring Deeply Nested Conditionals

Prompt:

Refactor the deeply nested if–else statements into a cleaner and more readable structure.Flatten the logic and improve maintainability.

Code:

```
#Refactor the deeply nested if-else statements into a cleaner and more readable s
"""def classify_number(num):
    Classify a number as positive, negative, or zero.
    if num > 0:
        return "Positive"
    elif num < 0:
        return "Negative"
    else:
        return "Zero"
def main():
    number = float(input("Enter a number: "))
    classification = classify_number(number)
    print(f"The number is classified as: {classification}")
if __name__ == "__main__":
    main()
"""
```

Output:

```
Enter a number: 12
The number is classified as: Positive
```

Significance:

Deep nesting decreases readability and increases cognitive complexity. Flattening the logic using guard clauses or mapping improves clarity, simplifies debugging, and enhances maintainability.

Task 3: Refactoring Repeated File Handling Code

Prompt:

Refactor repeated file open/read/close logic using a reusable function.Apply the DRY principle and use with open() context managers.

Code:

```
"""def read_file(file_path):
    Read the contents of a file and return it as a string.
    with open(file_path, 'r') as file:
        return file.read()
def main():
    file_path = input("Enter the path to the file: ")
    content = read_file(file_path)
    print("File contents:")
    print(content)
if __name__ == "__main__":
    main()

"""
```

Significance:

Repeated file open–read–close operations violate the DRY principle and increase error risk. Using a reusable function with context managers (with open) ensures automatic resource management, reduces duplication, and improves reliability.

Task 4: Optimizing Search Logic

Prompt:

Refactor the linear search to use a more efficient data structure like a set or dictionary.Improve time complexity and explain why the new approach is better.

Code:

```
"""def linear_search(arr, target):
    Perform a linear search for the target in the array.
    for index, value in enumerate(arr):
        if value == target:
            return index
    return -1
def main():
    arr = [1, 2, 3, 4, 5]
    target = int(input("Enter the number to search for: "))
    index = linear_search(arr, target)
    if index != -1:
        print(f"Number found at index: {index}")
    else:
        print("Number not found in the array.")
if __name__ == "__main__":
    main()
"""
```

Output:

```
Enter the number to search for: 4
Number found at index: 3
```

Significance:

Linear search requires O(n) time, which becomes inefficient for large datasets. Using sets or dictionaries provides average O(1) lookup time, improving performance and demonstrating appropriate data structure usage.

Task 5: Refactoring Procedural Code into OOP Design

Prompt:

Refactor the procedural salary calculation code into a class-based design.Apply object-oriented principles such as encapsulation and proper method usage

Code:

```
"""class Employee:
    def __init__(self, name, base_salary):
        self.name = name
        self.base_salary = base_salary
    def calculate_salary(self, bonus=0):
        return self.base_salary + bonus
def main():
    name = input("Enter employee name: ")
    base_salary = float(input("Enter base salary: "))
    bonus = float(input("Enter bonus amount: "))
    employee = Employee(name, base_salary)
    total_salary = employee.calculate_salary(bonus)
    print(f"Total salary for {employee.name} is: {total_salary:.2f}")
if __name__ == "__main__":
    main()
"""
```

Output:

```
Enter employee name: shryitha
Enter base salary: 100000
Enter bonus amount: 12000
Total salary for shryitha is: 112000.00
```

Significance:

Procedural salary computation lacks structure and scalability. Implementing a class encapsulates related data and behavior, promotes reusability, and aligns with object-oriented design principles.

Task 6: Refactoring for Performance Optimization

Prompt:

Refactor the loop-based summation code to improve performance.Use mathematical formulas or built-in functions instead of manual looping.

Code:

```
"""from xxlimited import new


def sum_of_natural_numbers(n):
    return n * (n + 1) // 2
def main():
    n = int(input("Enter a positive integer: "))
    total_sum = sum_of_natural_numbers(n)
    print(f"The sum of the first {n} natural numbers is: {total_sum}")
if __name__ == "__main__":
    main()"""
```

Output:

```
Enter a positive integer: 8
The sum of the first 8 natural numbers is: 36
```

Significance:

Iterative loops over large ranges can be computationally expensive. Using mathematical formulas or Python built-in constructs reduces time complexity, improves execution speed, and optimizes performance.

Task 7: Removing Hidden Side Effects

Prompt:

Refactor the code to remove hidden side effects caused by modifying global variables.Make the function return new values instead of mutating shared state.

Code:

```
"""def increment_counter(counter):
    return counter + 1
def main():
    counter = 0
    for _ in range(5):
        counter = increment_counter(counter)
    print(f"Final counter value: {counter}")
if __name__ == "__main__":
    main()
"""
```

Output:

```
Final counter value: 5
```

Significance:

Functions that modify shared mutable state introduce side effects and unpredictable behavior. Returning values instead of altering globals improves functional purity, testability, and program reliability

Task 8: Refactoring Complex Input Validation Logic

Prompt:

Refactor the nested password validation logic into separate validation functions.Improve readability, modularity, and testability.

Code:

```
"""def is_valid_length(password):
    return len(password) >= 8
def has_uppercase(password):
    return any(char.isupper() for char in password)
def has_lowercase(password):
    return any(char.islower() for char in password)
def has_digit(password):
    return any(char.isdigit() for char in password)
def validate_password(password):
    if not is_valid_length(password):
        return "Password must be at least 8 characters long."
    if not has_uppercase(password):
        return "Password must contain at least one uppercase letter."
    if not has_lowercase(password):
        return "Password must contain at least one lowercase letter."
    if not has_digit(password):
        return "Password must contain at least one digit."
    return "Password is valid."
def main():
    password = input("Enter a password: ")
    validation_result = validate_password(password)
    print(validation_result)
if __name__ == "__main__":
    main()"""
```

Output:

```
Enter a password: raju
Password must be at least 8 characters long.
```

Significance:

Nested validation conditions reduce readability and hinder testing. Separating validation rules into individual functions improves clarity, enables independent testing, and supports scalable validation logic.